

Fig. 7 | FPGA Implementation architecture and functional validation of the probabilistic building block. a The processing element architecture for a building block, as implemented on the FPGA. **b** Image samples generated by the FPGA with 1 probabilistic sample.

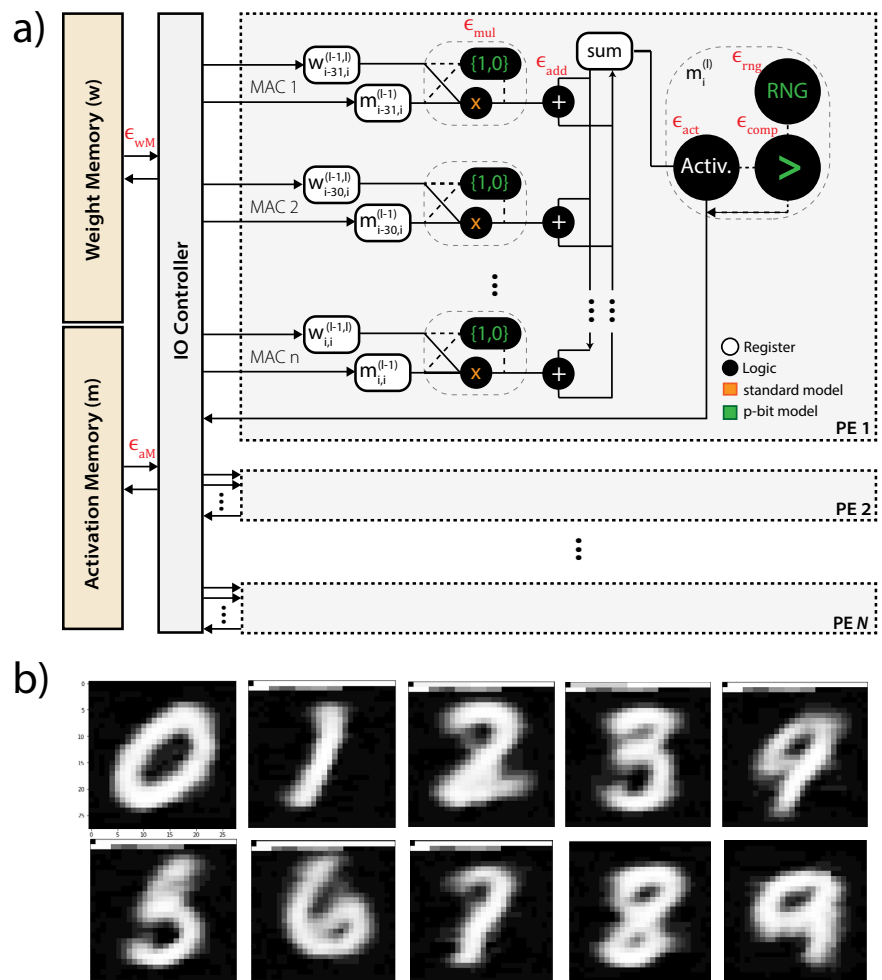


Table 2 | Energy and utilization for FPGA designs on the ZCU104

	Power (W)	Time (ns)	Energy (nJ)	LUTs	Regs
Full design					
p-bit	0.066	2.04e6	1.35e5	3796	2926
tanh	0.168	2.04e6	3.43e5	4158	2759
LFSR (RNG)					
p-bit	1.4e-4	–	–	16	33
Comparator					
p-bit	3.3e-4	–	–	17	33

- Only fully connected layers are implemented, as opposed to convolutional layers that may require sliding windows.
- The fan-in to any neuron in the network is limited to 32, similar to how, e.g., k-nearest neighbor designs for Boltzmann machines limit fan-in to k.

The image generation task is implemented via a DNN based on a Variational Autoencoder, shown in Fig. 9c, trained on the MNIST dataset. This DNN is mapped onto a hardware processing element as described by Fig. 7a. Two versions of this DNN are trained, one that uses p-bits and one that uses standard 32b σ activations, and consequently, two versions of the hardware are built. There are 2 differences between the designs. First, the *synapse* module implements a MAC operation for the standard design as opposed to AC for the p-bit design. Second, the *neuron* module skips the RNG and comparison operations for the standard design. These changes are summarized in Fig. 7.

The power and energy of these designs are compared in Table 2. We observe a 4.42x improvement in the energy consumption of the mac module alone, which leads to an overall energy improvement of 2.55x. This aligns with the gain prediction in Fig. 8, where a 1.875 pJ/bit memory energy (approximate energy/bit for BRAM on the ZU7EV fpga) corresponds to 2.3x gain, aligning closely with our experimentally obtained gain of 2.5x. With more efficient memory, this gain could be much larger as shown in Fig. 8b, c, motivating the incorporation of in-memory-compute architectures. We also observe that mac-related CLB LUT usage goes down by 2.9x.

A key consideration of using probabilistic models is the overhead that comes from the RNG over deterministic models. We see that the RNG and comparison overhead are negligible in terms of energy and area compared to the rest of the inference pipeline in custom hardware, which will only grow smaller as n grows larger.

While the absolute gains observed on FPGA are conservative by design, they validate the scaling trends predicted by the energy framework, which become increasingly favorable as models grow larger and memory energy dominates, as in contemporary LLM-scale workloads. We also note that the FPGA implementation is intended to validate the energy decomposition and scaling behavior predicted by the analytical framework, rather than to benchmark accuracy. Functional correctness was verified through generated outputs, while accuracy comparisons are reported using high-precision GPU simulations to avoid hardware-specific numerical confounds.

Discussion

The sample-aware training algorithm, paired with the energy analysis in Fig. 6, opens avenues to interesting probabilistic compute architectures that could optimize for latency or energy efficiency. One possibility is the

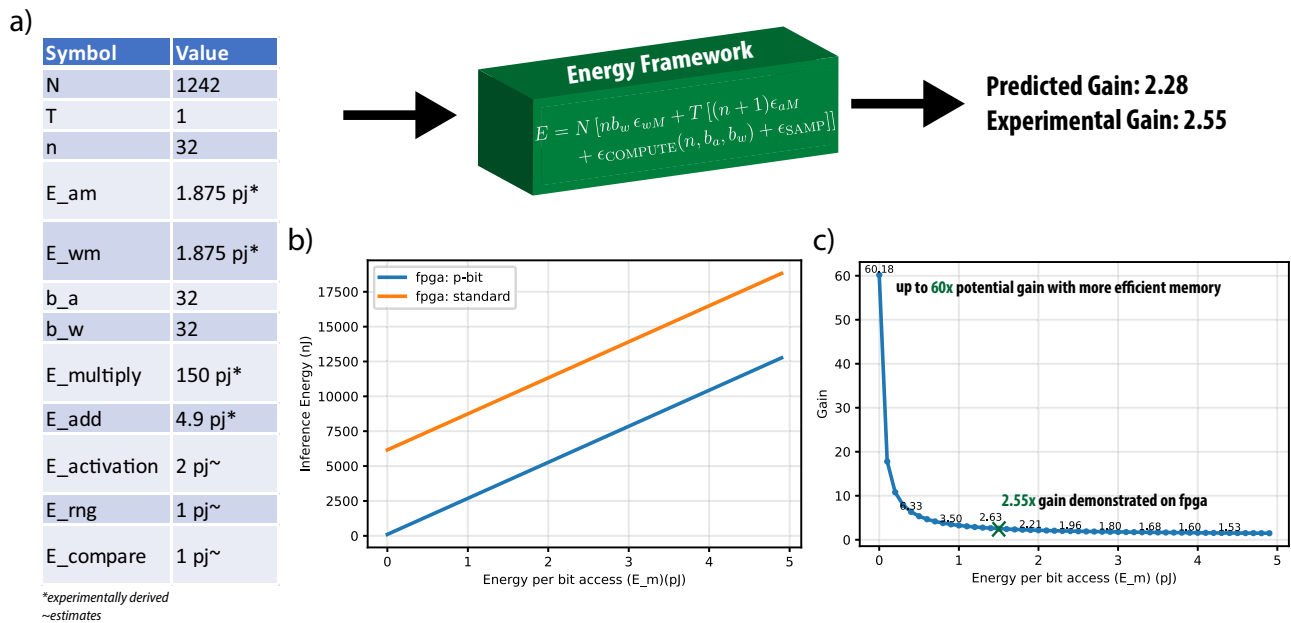


Fig. 8 | Energy analysis for FPGA Implementation. **a** Contains values used for the FPGA design's energy prediction. **b** Is the predicted energy per frame of inference on the FPGA. **c** Shows the predicted gain curve comparing probabilistic and deterministic models across changing memory efficiency ($E_{wM} = E_{aM} = E_M$).

drawing of samples in parallel across separate hardware units, so long as each sample is independent and identically distributed (i.i.d), which they are for these models, and combining the samples at the output of the last layer. The internal math is all 1-bit, but the result could be the equivalent of 3-bit accuracy, delivered in less time. Such a hardware system could be a perfect fit for latency-sensitive situations where energy and resources are not a concern. This is an opportunity that quantization cannot take advantage of.

Another possibility for taking advantage of sampling is to build an n -bit p-DNN, which would match the energy of an n -bit DNN to within the difference in neuron energy ($\epsilon_{\text{rng}} + \epsilon_{\text{compare}}$). Adding an additional sample to the inference pipeline could lead to 2+ % in accuracy, while increasing energy by just 0.7% (see Fig. 6b) for models that run on DRAM. We consider our estimate conservative in that larger, production AI models, like LLMs, would use significantly more memory-related energy, making the additional sampling energy minuscule.

An important advantage to doing inference via probabilistic systems over direct quantization is the ability to adjust the energy and accuracy at run-time through the number of samples. We show that a 1-bit probabilistic inference engine can be the equivalent of a 1-bit, 2-bit, and 3-bit quantized inference engine rolled into one. While each comparison on its own leads to roughly parity energy use, this is a giant practical leap, as a single hardware can serve all 3 bit-widths.

These unique aspects of probabilistic inference extend from the foundation of probabilistic computing. The FPGA experiment embodies the connection of this work to its motivating foundations. It shows that the same building block used for Boltzmann machines can implement p-DNNs. The idea is to show that p-DNNs can be implemented on near-BM hardware, for which there has already been extensive work done to build devices for and optimize at the system level^{7,9,10}.

How is it that machinery designed for Boltzmann machines can be applied to feed-forward DNNs? Graph colored Ising models, for instance, enforce updating a subset of the total neurons per timestep such that every neuron in a model with N_c colors would have an attempt to flip in N_c timesteps. The neurons of each color could be thought of as a layer in a feed-forward neural network; an MLP with p-bit activations could be seen as a rearranged graph-colored Ising model. In the same way, Ising models with limited connectivity per neuron are able to be mapped to efficient in-memory-compute architectures¹¹, limited connectivity MLPs could be mapped to in-memory-computing architectures for ML tasks. The right

balance between connectivity and performance could be application-specific. An important benefit to implementing p-bits in a feed-forward fashion is that the backpropagation algorithm can be used to train the model, as opposed to standard energy-based training schemes like Hebbian learning or contrastive divergence, which are notably less efficient.

While preliminary works into probabilistic machine learning exist, they are not motivated by mapping their designs to unconventional computing substrates, and importantly, do not take advantage of sampling during inference. While¹² state that stochastic inference surpasses HP models within 25 samples for CIFAR10 classification, we find that accuracy comes to within 1% but does not quite match. On the other hand, models implementing deterministic inference have found great success. For vision applications, W1A1 models have proven to be quite commendable^{2,13,14}. Most recently,¹⁵ have set state-of-the-art results using W1A1 models for vision applications. Stochastic inference for feed-forward CNNs was studied in¹² but we were not able to replicate their results.

While the experiments shown in this work demonstrate probabilistic inference on vision models, the ideas, and the presented energy framework, easily extend to Large Language Models (LLMs). Feed-forward machine learning models are at the core of what makes AI so powerful today. While high bit weight and activation models (W32A32) were common a decade ago, the energy cost of running today's AI models has skyrocketed. The energy benefits of going to $W \times A_y$ models where x and y are small, are clear, making it a highly active area of research. Recent works have reduced weight precision in large language models (LLMs) to 4 bits and beyond^{16,17}, but activation precision has remained challenging to reduce below 8 bits due to accuracy falloff. More so than weights, activations need to be adaptable at runtime, able to handle large-scale outliers that may arise during inference^{18,19}. This is where p-bits may provide a means to scale down LLM activation bits without compromising on accuracy. A 4-bit activation LLM for instance may achieve the required accuracy using a few samples. With fine-tuning and retraining methods like sample-aware training, that accuracy may be boosted, motivating research in that direction. While this work focuses on inference energy, extending sample-aware training to reduce training-time energy—particularly for large language models—is an important direction for future work and is not claimed as a contribution here.

We'd like to note that, as it stands, the energy efficiency of p-DNNs, similar to standard DNNs, will depend more on memory technology

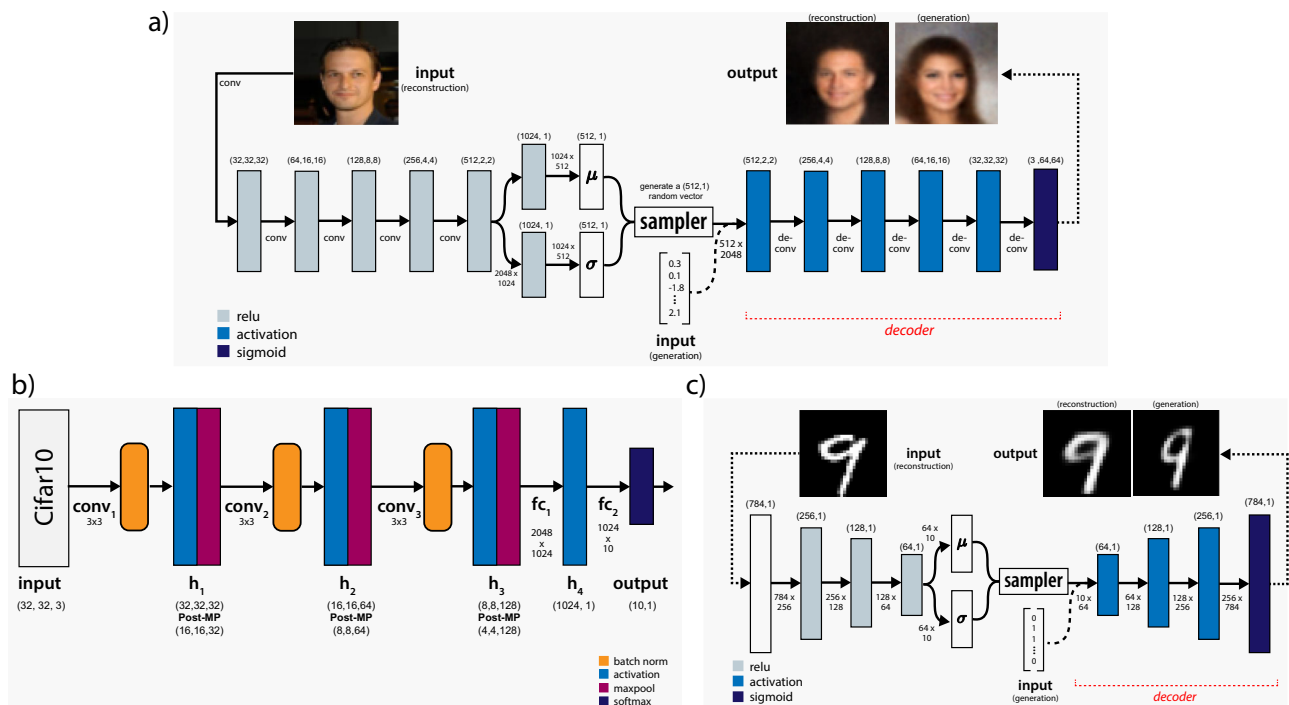


Fig. 9 | Architectures of models used for image generation and classification experiments. a Model for celebrity image generation experiment. **b** Model for CIFAR10 classification experiment. **c** Model for MNIST generation FPGA experiment.

compared to changes in other facets (tech-node, model architecture, etc.). In order for p-bits to outperform standard quantization on a purely *compute energy* front (synapse + neuron energy), advancements in sampling-based training schemes and implementations that enable drawing fewer samples will be critical. One example of this is finding more effective ways to combine different samples, beyond simple averages. The flip side to this, as we noted previously, is that additional compute on the same data, in the vein of sampling, can improve accuracy without any notable impact on end-to-end energy.

P-bits are a versatile tool with a proven track record in building highly energy-efficient platforms for sampling and optimization problems, now applied to DNNs. We discuss two compute-only sampling schemes that can improve on the accuracy of standard DNNs with minor end-to-end energy penalties, demonstrated on image classification and generation. We pair this with an energy framework for probabilistic systems based on the basic building block consisting of a p-bit and its synapse. This building block is mapped to the p-bit-based ASIC in ref. 1 to reveal its energy components. We further support our framework with an FPGA demonstration and show closely matching energy improvements to the framework's prediction. Conceptually, this work establishes that probabilistic computing primitives—previously confined to energy-based and optimization-centric models—can be systematically extended to feedforward deep learning, enabling a unified hardware substrate for both sampling-based inference and gradient-trained neural networks. Future work could study analog implementations for p-bit-based DNNs, integrating with in-memory compute architectures, or extending these results beyond vision applications.

Methods

The following are simulation details for training the DNNs used in this manuscript. Namely, this includes the classification model and two image generation models, one of which was run on the FPGA. Code is available on request.

For classification experiments, we went with sigmoid for the p-bit implementation to give deterministic models the benefit of the doubt, as

sigmoid activations performed much better for deterministic models. For image generation, we found no notable difference in performance between sigmoid and tanh, and so stuck with tanh p-bits and tanh deterministic baselines. For each experiment, the derivative of the p-bit was implemented as the derivative of the activation, going “straight-through” the sign function²⁰. All models were trained on an RTX 4070 Super, and Wandb was used to log and track experiments.

Classification experiment

The classification experiment was carried out on the CIFAR-10 dataset. This dataset contains (32,32,3) dimensional images that span 10 classes: plane, car, bird, cat, deer, dog, frog, horse, ship, truck. The data is split between 50000 training images and 10000 test images.

The architecture is depicted in Fig. 9. All models are trained for 1000 epochs using the ADAM optimizer on cross-entropy loss using pytorch. All models use a batch size of 64 and have a constant learning rate of 1e-3. There is a standard horizontal flip pre-processing on the dataset during training for all models. Batch norm can be folded into the preceding convolutional or fully connected layer during inference²¹, allowing us to take advantage of it during training without any penalty on inference energy.

The CIFAR10-classification models are initialized with weights following Glorot (Xavier) weight initialization from ref. 22. Glorot initialization is a one-time overhead that ensures that the weights of the model have the same variance per layer despite changing layer configurations, and are centered around 0. It can be described as follows.

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{\text{fan_in} + \text{fan_out}}\right) \quad (5)$$

where i and j are the input and output units of weight matrix W . fan_in describes the input size of weight layer W and fan_out describes the output size. Biases are set to 0.